DD: FUL-04 v1.1 — Restriction Fulfillment

Developer implementation guide: `DD_FUL-04-Restriction_Fulfillment-DEV_IMPL_GUIDE-v1.1.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_FUL-04-Restriction_Fulfillment-v1.1.md`.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Fulfillment | **Version:** v1.1 Orchestrates the technical fulfillment of "restrict / suspend service" intents on a customer's active services. Called synchronously by SUB-WF-RESTRICT-01 (apply/lift restriction) and SUB-WF-SUSPEND-NP-01 (non-payment suspension) via the framework's `ful-call-restrict` / `ful-call-suspend` workers. FUL-04 loads the Package's Services (per SIP-01 + PLM-CFG-01), iterates each Service, resolves the network target via the HomePass's `service_management_endpoints` map (per RLM-CFG-01), and dispatches to the Provisioner adapter — using `RestrictionCapableProvisioner` for APPLY_RESTRICTION / LIFT_RESTRICTION verbs and `ServiceProvisioner.suspendService` for SUSPEND_SERVICE. Aggregates outcomes, supports compensation on partial failure, returns a structured outcome. **What changed from v1.0:** - **Aligned with PLM-CFG-01 v1.0's formal Provisioner interfaces.** APPLY_RESTRICTION / LIFT_RESTRICTION dispatch through `RestrictionCapableProvisioner` (PLM-CFG-01 §3.4 extension interface). SUSPEND_SERVICE dispatches through `ServiceProvisioner.suspendService` (base interface, PLM-CFG-01 §3.1). RESUME_SERVICE is now FUL-03 v1.1's responsibility (not FUL-04). - **Operator scoping** — every envelope carries `operatorCode`. Catalog reads (PLM-CFG-01 Services, RLM-CFG-01 HomePass endpoints) filter by operator. Provisioner adapter lookup is per `(provisionerKey, operatorCode)`. - `RestrictionCapableProvisioner` **capability check** — before dispatching APPLY/LIFT, FUL-04 verifies the registered adapter implements `RestrictionCapableProvisioner`. If not (e.g., partner-fulfilled OTT bundle adapter), the per-Service outcome records `RESTRICTION_NOT_SUPPORTED_BY_PROVISIONER` and the Service is skipped from the chain. Per PLM-CFG-01 R-PR-3. - **Section cleanup** — removed "C — V3 design intent", "Migration approach", "Open coordination items" sections per memory rule #26.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-04-Restriction_Fulfillment-v1.1.md`
Version	`v1.1`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `POST /api/restriction-fulfillment/execute`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `ful-call-restrict`
- `ful-call-suspend`
- `gpon-res`
- `notification-send`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Fulfillment | **Version:** v1.1

Orchestrates the technical fulfillment of "restrict / suspend service" intents on a customer's active services. Called synchronously by SUB-WF-RESTRICT-01 (apply/lift restriction) and SUB-WF-SUSPEND-NP-01 (non-payment suspension) via the framework's `ful-call-restrict` / `ful-call-suspend` workers. FUL-04 loads the Package's Services (per SIP-01 + PLM-CFG-01), iterates each Service, resolves the network target via the HomePass's `service_management_endpoints` map (per RLM-CFG-01), and dispatches to the Provisioner adapter — using `RestrictionCapableProvisioner` for APPLY_RESTRICTION / LIFT_RESTRICTION verbs and `ServiceProvisioner.suspendService` for SUSPEND_SERVICE. Aggregates outcomes, supports compensation on partial failure, returns a structured outcome.

What changed from v1.0:

- **Aligned with PLM-CFG-01 v1.0's formal Provisioner interfaces.** APPLY_RESTRICTION / LIFT_RESTRICTION dispatch through `RestrictionCapableProvisioner` (PLM-CFG-01 §3.4 extension interface). SUSPEND_SERVICE dispatches through `ServiceProvisioner.suspendService` (base interface, PLM-CFG-01 §3.1). RESUME_SERVICE is now FUL-03 v1.1's responsibility (not FUL-04).

- **Operator scoping** — every envelope carries `operatorCode`. Catalog reads (PLM-CFG-01 Services, RLM-CFG-01 HomePass endpoints) filter by operator. Provisioner adapter lookup is per `(provisionerKey, operatorCode)`.

- `RestrictionCapableProvisioner` **capability check** — before dispatching APPLY/LIFT, FUL-04 verifies the registered adapter implements `RestrictionCapableProvisioner`. If not (e.g., partner-fulfilled OTT bundle adapter), the per-Service outcome records `RESTRICTION_NOT_SUPPORTED_BY_PROVISIONER` and the Service is skipped from the chain. Per PLM-CFG-01 R-PR-3.

- **Section cleanup** — removed "C — V3 design intent", "Migration approach", "Open coordination items" sections per memory rule #26.

Glossary

Term	Meaning
Intent	The verb FUL-04 is executing: `APPLY_RESTRICTION` / `LIFT_RESTRICTION` / `SUSPEND_SERVICE`. Carried in the request envelope.
`APPLY_RESTRICTION`	Apply a partial network-level restriction (e.g., bar outgoing calls, throttle data speed). Carried by SUB-WF-RESTRICT-01 with a `restrictionCode`. Reversible via LIFT_RESTRICTION.
`LIFT_RESTRICTION`	Remove a previously-applied restriction. Carries `remainingActiveRestrictions[]` so the adapter computes the net new effective state.
`SUSPEND_SERVICE`	Apply a network-level non-payment suspension. Carried by SUB-WF-SUSPEND-NP-01 + by package-change workflows (UPGRADE/DOWNGRADE/MIGRATION/RELOCATION) at their source HomePass during commit window. The inverse is FUL-03 v1.1's `RESUME_SERVICE` — NOT a FUL-04 verb.

Term	Meaning
`restrictionCode`	A code identifying which restriction to apply or lift. Operator-defined enumeration (e.g., `OUTGOING_VOICE_BARRED`, `DATA_THROTTLED`, `INCOMING_VOICE_BARRED`). Adapter-specific semantics.
`fulfillmentAction`	A field on the restriction-handling envelope indicating the operator's policy for this specific restriction action (e.g., immediate, deferred-to-cycle-end). Adapter interprets per policy.
Service management endpoints	The `service_management_endpoints` map on the HomePass row (RLM-CFG-01-owned): `{serviceManagementKey → {nodeCode, port}}`. FUL-04 resolves each Service's target via this map.
Provisioner adapter	Code module implementing `ServiceProvisioner` (and optionally `RestrictionCapableProvisioner`) — registered per `(provisionerKey, operatorCode)`. Translates FUL-04 verbs into vendor commands.
`RestrictionCapableProvisioner`	The PLM-CFG-01 v1.0 §3.4 extension interface declaring `applyRestriction` and `liftRestriction` methods. Adapters that support restrictions implement it. Adapters that don't (e.g., partner OTT) don't, and APPLY/LIFT calls on their Services are recorded as `RESTRICTION_NOT_SUPPORTED_BY_PROVISIONER`.

1. What this DD owns

1. **The `ful-call-restrict` and `ful-call-suspend` worker contracts** — canonical request/response envelopes.
2. **Catalog resolution** — given subscription + Package + HomePass, resolve which Services to dispatch.
3. **Per-Service dispatch** — invoke the right Provisioner method per intent.
4. **Compensation** — undo previously-succeeded Services on chain failure.
5. **Outcome aggregation** — per-Service outcomes rolled into a terminal state.
6. **Domain Kafka events** — `RestrictionFulfillmentStarted` / `Completed` / `Failed`, `CompensationFailed`.

What this DD does NOT own

- **Restriction lifecycle / which codes apply when** — SUB-WF-RESTRICT-01.
- **Non-payment suspension orchestration** — SUB-WF-SUSPEND-NP-01.
- **Resume / un-suspend orchestration** — FUL-03 v1.1's `RESUME_SERVICE` verb. The resume path is a fresh FUL-03 call by SUB-WF-RESUME-01; FUL-04 has no RESUME verb (its only "resume" usage is internal compensation inverse of SUSPEND_SERVICE on chain failure).
- **Package change orchestration** — SUB-WF-UPGRADE-01 / DOWNGRADE-01 / MIGRATION-01 / RELOCATION-01 (they call FUL-04 for source-side SUSPEND_SERVICE during commit window, then FUL-03 for target-side ACTIVATE).
- **Termination network revocation** — FUL-05.
- **HomePass row mutations** — RLM-CFG-01 is read-only from FUL-04's perspective.
- **HomePass bookkeeping (CLAIM / RELEASE)** — FUL-07.
- **Notifications** — NOT-01.

Operator scoping

Every FUL-04 call carries `operatorCode`. Catalog reads (PLM-CFG-01 Services, RLM-CFG-01 HomePass) filter by operator. The Provisioner adapter registry is keyed per `(provisionerKey, operatorCode)` — same `provisionerKey` may have different adapter implementations across operators (e.g., one operator's `gpon-res` adapter targets Huawei NCE while another's targets a different vendor).

2. Canonical envelope

2.1 Request (APPLY_RESTRICTION)

```
POST /api/restriction-fulfillment/execute HTTP/1.1
Content-Type: application/json

{
```

```
  "operatorCode":             "WIK",
  "subscriptionId":           "sub_01HZ_KAMAU_FIBER",
  "customerId":               "cust_01HZ_KAMAU",
  "packageRef":               "pkg_01HX_INET_100M_VOIP_RES",
  "packageVersionId":         "pkv_01HX_INET_100M_V3",
  "homepassId":               "hp_01HZ_KAREN_017",
  "intent":                   "APPLY_RESTRICTION",
  "restrictionCode":          "OUTGOING_VOICE_BARRED",
  "fulfillmentAction":        "IMMEDIATE",
  "fulfillmentCorrelationId": "subop_01HZ_KAMAU_RESTRICT:apply",
  "callerWorkflowKind":       "SUB_WF_RESTRICT",
  "callerWorkflowRefId":      "subop_01HZ_KAMAU_RESTRICT"
}
```

2.2 Request (LIFT_RESTRICTION)

```
{
  "operatorCode":              "WIK",
  ...
  "intent":                    "LIFT_RESTRICTION",
  "restrictionCode":           "OUTGOING_VOICE_BARRED",
  "fulfillmentAction":         "IMMEDIATE",
  "remainingActiveRestrictions": ["DATA_THROTTLED"],
  "fulfillmentCorrelationId":  "subop_01HZ_KAMAU_RESTRICT:lift"
}
```

2.3 Request (SUSPEND_SERVICE)

Called by SUB-WF-SUSPEND-NP-01 (non-payment) and by package-change workflows (UPGRADE/DOWNGRADE/MIGRATION/RELOCATION) at source HomePass during commit window.

```
{
  "operatorCode":              "WIK",
  ...
  "intent":                    "SUSPEND_SERVICE",
  "reasonCategory":            "NON_PAYMENT",
  "dunningEscalationLevel":    3,
  "fulfillmentCorrelationId":  "subop_01HZ_KAMAU_SUSPEND_NP:suspend"
}
```

For package-change workflows calling SUSPEND_SERVICE at source HomePass: `reasonCategory = SOURCE_DECOMMISSION` (or operator-specific equivalent); `dunningEscalationLevel` is omitted (NULL).

2.4 Response

```
{
  "fulfillmentId":             "fl4b_01HZ_KAMAU_APPLY",
  "operatorCode":              "WIK",
  "subscriptionId":            "sub_01HZ_KAMAU_FIBER",
  "intent":                    "APPLY_RESTRICTION",
  "restrictionCode":           "OUTGOING_VOICE_BARRED",
  "terminalState":             "FULLY_FULFILLED",
  "serviceOutcomes": [
    {
      "serviceRef":           "svc_VOIP_STD",
      "outcome":              "SUCCEEDED",
      "dispatchedVerb":       "applyRestriction",
      "targetNodeCode":       "VOIPSWITCH-NRB-01",
      "targetPort":           "5012",
      "adapterRefs":          {"sipBarringId": "bar_77123"},
      "durationMs":           450
    },
    {
      "serviceRef":           "svc_INET_100M",
      "outcome":              "SKIPPED_NOT_APPLICABLE",
      "dispatchedVerb":       null,
      "skipReason":           "OUTGOING_VOICE_BARRED does not affect INET service"
    }
  ],
  "durationMs":                450,
  "completedAt":               "2026-05-15T10:32:45+03:00"
}
```

3. Rules

3.1 Entry points

Rule	Description
R-FUL-04-EN-1	FUL-04 v1.1 exposes ONE primary endpoint: `POST /api/restriction-fulfillment/execute`. The request envelope carries `intent ∈ {APPLY_RESTRICTION, LIFT_RESTRICTION, SUSPEND_SERVICE}`.
R-FUL-04-EN-2	Required fields: `operatorCode`, `subscriptionId`, `customerId`, `packageRef`, `packageVersionId`, `homepassId`, `intent`, `fulfillmentCorrelationId`, `callerWorkflowKind`, `callerWorkflowRefId`. Missing any → 400 `MISSING_REQUIRED_FIELD`.
R-FUL-04-EN-3	For `intent = APPLY_RESTRICTION` or `LIFT_RESTRICTION`: also required are `restrictionCode` and `fulfillmentAction`. For `LIFT_RESTRICTION` only: also required `remainingActiveRestrictions[]`.
R-FUL-04-EN-4	For `intent = SUSPEND_SERVICE`: also required is `reasonCategory`. Optional: `dunningEscalationLevel` (present for non-payment suspensions, absent for package-change source-side decommission).
R-FUL-04-EN-5	Idempotency: same `(subscriptionId, fulfillmentCorrelationId)` re-submitted within 24h returns the prior ledger row's outcome with HTTP 200. Different `fulfillmentCorrelationId` for the same subscription within 24h with a prior fulfillment in flight returns 409 `CONCURRENT_FULFILLMENT_IN_PROGRESS`.
R-FUL-04-EN-6	Time budget: 60 seconds. Mid-orchestration timeout halts further dispatch; in-flight Service completes (its own timeout governs); chain treated as partial failure subject to compensation per policy.

3.2 Load + resolve

Rule	Description
R-FUL-04-LO-1	FUL-04 loads the Package via SIP-01 using `packageRef` + `packageVersionId` filtered by `operatorCode`. Reads `serviceRefs[]`. If not found → `PACKAGE_NOT_FOUND`.
R-FUL-04-LO-2	For each Service ref, loads the Service def via PLM-CFG-01 filtered by `operatorCode`. Reads `consumption_model`, `is_addressable`, `network_profile_shape`, `service_management_key`, `provisioner_key`. Non-addressable or non-CONTINUOUS Services are skipped (`outcome = SKIPPED_NOT_ADDRESSABLE`).
R-FUL-04-LO-3	Loads the HomePass via RLM-CFG-01 (read-only) using `homepassId`. Reads `service_management_endpoints` + `status_code`. For `APPLY_RESTRICTION`: HomePass in a state blocking activation per RLM-CFG-01's rules (e.g., not RFS / ACT) fails with `HOMEPASS_NOT_READY`. For `LIFT_RESTRICTION` and `SUSPEND_SERVICE` on a non-ready HomePass, the chain proceeds (Service outcomes record what adapters report).
R-FUL-04-LO-4	For each Service whose `service_management_key` is non-null, resolves target via `homepass.service_management_endpoints[service.service_management_key]` → `{nodeCode, port}`. Missing key → `NO_HOMEPASS_ENDPOINT_FOR_KEY` (Service-level failure). Null port → `HOMEPASS_PORT_NOT_CAPTURED` (Service-level failure).
R-FUL-04-LO-5	For each Service with null `service_management_key` (partner-fulfilled), passes `(null, null)` to the adapter.

3.3 Adapter capability check (APPLY_RESTRICTION / LIFT_RESTRICTION only)

Rule	Description
R-FUL-04-CC-1	Before dispatching `APPLY_RESTRICTION` or `LIFT_RESTRICTION`, FUL-04 looks up the Provisioner adapter by `(provisioner_key, operatorCode)` and verifies it implements `RestrictionCapableProvisioner` (PLM-CFG-01 §3.4).
R-FUL-04-CC-2	If the adapter does NOT implement `RestrictionCapableProvisioner`: per-Service outcome recorded as `RESTRICTION_NOT_SUPPORTED_BY_PROVISIONER`. Service is skipped from dispatch (no Provisioner call made). Other Services in the chain proceed independently.
R-FUL-04-CC-3	`SUSPEND_SERVICE` dispatch does NOT require `RestrictionCapableProvisioner` — it uses the base `ServiceProvisioner.suspendService` method (PLM-CFG-01 §3.1) which all adapters implement. Adapters that cannot suspend (rare) return `success=false, failureReason=NOT_SUPPORTED` per PLM-CFG-01 R-PR-2; this manifests as a per-Service `outcome = FAILED` rather than `SKIPPED`.

3.4 Per-service dispatch

Rule	Description
R-FUL-04-DI-1	Services dispatched in the order declared by `Package.serviceRefs[]` (or operator-policy override). For SUSPEND_SERVICE, the recommended ordering is voice first, then data — drops the customer's voice line first while keeping data accessible (so the customer can reach payment portal mid-suspension). The Package owner declares the order; FUL-04 honors it.
R-FUL-04-DI-2	For `intent = APPLY_RESTRICTION`: invokes `RestrictionCapableProvisioner.applyRestriction(fulfillmentAction, restrictionCode, params, customer, targetNodeCode, targetPort)`. `params` is `service.networkProfileShape.params`. Returns `ProvisionerOutcome`.
R-FUL-04-DI-3	For `intent = LIFT_RESTRICTION`: invokes `RestrictionCapableProvisioner.liftRestriction(fulfillmentAction, restrictionCode, params, customer, targetNodeCode, targetPort, remainingActiveRestrictions[])`. The adapter computes the net effective state from `remainingActiveRestrictions[]`. Returns `ProvisionerOutcome`.
R-FUL-04-DI-4	For `intent = SUSPEND_SERVICE`: invokes `ServiceProvisioner.suspendService(reasonCategory, params, customer, targetNodeCode, targetPort)`. Returns `ProvisionerOutcome`.
R-FUL-04-DI-5	Per-service `SKIPPED_NOT_APPLICABLE` outcome is valid: when a `restrictionCode` doesn't apply to a Service (e.g., `OUTGOING_VOICE_BARRED` on an INET-only Service), the adapter returns `success=true, skipped=true`; FUL-04 records `outcome = SKIPPED_NOT_APPLICABLE`. The chain continues.

3.5 Compensation

Rule	Description
R-FUL-04-CO-1	Compensation invoked when terminal state is `FAILED_AND_ROLLED_BACK`. For each previously-successful Service, FUL-04 calls the inverse: `applyRestriction` → `liftRestriction(remainingActiveRestrictions = pre-chain state)`; `liftRestriction` → `applyRestriction(restrictionCode, fulfillmentAction)`; `suspendService` → `resumeService()` (the SUSPEND_SERVICE inverse, used internally for compensation only — NOT a FUL-04 entry-point verb). Order: reverse of original dispatch.
R-FUL-04-CO-2	The `ServiceProvisioner.resumeService` method (PLM-CFG-01 §3.1) is the SUSPEND_SERVICE compensation inverse. Legitimate resume (post-pause / post-non-payment-suspend) is FUL-03 v1.1's `RESUME_SERVICE` entry point, not a FUL-04 endpoint. The shared interface method serves both consumers cleanly.
R-FUL-04-CO-3	If compensation itself fails for any Service: terminal compensation state recorded as `COMPENSATION_FAILED` in the ledger; emit `CompensationFailed` Kafka event; NOC recovery required.

3.6 Outcome aggregation

Rule	Description
R-FUL-04-OA-1	Terminal state = `FULLY_FULFILLED` when all dispatched Services succeed or skip.
R-FUL-04-OA-2	Terminal state = `PARTIALLY_FULFILLED` when some Services succeed and some fail BUT the failure mode is `policy = TOLERATE_PARTIAL`. The caller decides whether to accept or revert.
R-FUL-04-OA-3	Terminal state = `FAILED_AT_FIRST_SERVICE` when the first Service fails AND `policy = ABORT_ON_FIRST_FAIL` (default). No compensation needed.
R-FUL-04-OA-4	Terminal state = `FAILED_AND_ROLLED_BACK` when a Service fails AFTER prior successes AND policy requires rollback. Compensation runs per §3.5.
R-FUL-04-OA-5	Per-Service outcomes recorded in `fulfillment_service_outcome` (one row per Service per fulfillment). Carries `service_ref`, `outcome` (`SUCCEEDED` / `FAILED` / `SKIPPED_NOT_APPLICABLE` / `SKIPPED_NOT_ADDRESSABLE` / `RESTRICTION_NOT_SUPPORTED_BY_PROVISIONER` / `COMPENSATED` / `COMPENSATION_FAILED`), `adapter_refs`, `failure_reason`, `failure_detail`, `target_node_code`, `target_port`, `duration_ms`.

3.7 Operator scoping

Rule	Description
R-FUL-04-OP-1	Every envelope carries `operatorCode`. Catalog reads (PLM-CFG-01 Service, RLM-CFG-01 HomePass, SIP-01 Package) filter by operator. Cross-operator references rejected with 400.
R-FUL-04-OP-2	Provisioner adapter registry is keyed per `(provisionerKey, operatorCode)`. Same `provisionerKey` may have different implementations across operators. The `@ServiceProvisioner(key = "...", operatorCode = "...")` Spring annotation registers each adapter under its operator.
R-FUL-04-OP-3	Operator-specific restriction codes live in PLM-CFG-01 / RLM-CFG-01 catalogs (operator-scoped). FUL-04 does NOT define a canonical cross-operator `restrictionCode` enumeration — each operator declares its own; SUB-WF-RESTRICT-01 supplies the code; FUL-04 passes through to the adapter.

4. Worked example — Kamau gets voice-barred via dunning

Context: Kamau is on `pkg_INET_100M_VOIP_RES` (double-play) at HomePass `hp_01HZ_KAREN_017`. BIL-04 dunning escalates to level 2 → calls SUB-WF-RESTRICT-01 with `restrictionCode = OUTGOING_VOICE_BARRED, fulfillmentAction = IMMEDIATE`. SUB-WF-RESTRICT-01 validates + applies the restriction state + reaches `callFulfillment`.

10:32:42 — `ful-call-restrict` **worker invokes FUL-04.** Worker POSTs to `/api/restriction-fulfillment/execute` with the APPLY_RESTRICTION envelope (§2.1).

10:32:42.1 — Idempotency + ledger init. FUL-04 checks `restriction_fulfillment_ledger` for `(subscriptionId, fulfillmentCorrelationId)`. No prior row — inserts `fl4b_01HZ_KAMAU_APPLY` with `terminal_state = NULL`.

10:32:42.2 — Load. FUL-04 calls SIP-01 → Package serviceRefs `[svc_INET_100M, svc_VOIP_STD]`. PLM-CFG-01 → INET Service (`provisioner_key=gpon-res, service_management_key=data`) and VOIP Service (`provisioner_key=voip-sip, service_management_key=voice`). RLM-CFG-01 → HomePass `hp_01HZ_KAREN_017`, `service_management_endpoints = {data: {OLT-NRB-KAREN-02, 0/4/07}, voice: {VOIPSWITCH-NRB-01, 5012}}`, `status=ACT`.

10:32:42.3 — Adapter capability check. For INET service: looks up `(gpon-res, WIK)` adapter → `GponResProvisioner` bean → checks `instanceof RestrictionCapableProvisioner`. GPON adapter implements it (supports speed throttling). For VOIP service: looks up `(voip-sip, WIK)` → `VoipSipProvisioner` → implements `RestrictionCapableProvisioner` (supports call barring).

10:32:42.4 — Dispatch INET. Calls `gponResProvisioner.applyRestriction(fulfillmentAction="IMMEDIATE", restrictionCode="OUTGOING_VOICE_BARRED", params={speedDownMbps: 100, ...}, customer, "OLT-NRB-KAREN-02", "0/4/07")`. The adapter examines the restrictionCode — `OUTGOING_VOICE_BARRED` doesn't affect INET — returns `success=true, skipped=true, skipReason="OUTGOING_VOICE_BARRED does not affect INET service"`. FUL-04 records `outcome = SKIPPED_NOT_APPLICABLE`. Chain continues.

10:32:42.5 — Dispatch VOIP. Calls `voipSipProvisioner.applyRestriction(fulfillmentAction="IMMEDIATE", restrictionCode="OUTGOING_VOICE_BARRED", params={codec: "G.711a", ...}, customer, "VOIPSWITCH-NRB-01", "5012")`. The adapter calls the SIP switch via vendor API: applies a class-of-service profile that blocks outbound origination from Kamau's DID, keeps inbound and emergency outbound enabled per regulatory carve-out. Returns `success=true, adapterRefs={sipBarringId: "bar_77123"}` after 450ms. FUL-04 records `outcome = SUCCEEDED`.

10:32:42.6 — Outcome aggregation. 2 services dispatched (1 SKIPPED_NOT_APPLICABLE, 1 SUCCEEDED). Terminal state `FULLY_FULFILLED`. FUL-04 updates ledger row + emits `RestrictionFulfillmentCompleted` to `sophix.fulfillment.restriction.completed`. Returns 200 to caller.

10:32:42.7 — Worker completes Camunda task. SUB-WF-RESTRICT-01's BPMN advances; the saga's next step is `notification-send` (NOT-01 customer notification — "Your outgoing calls are temporarily restricted pending payment").

Total elapsed time: 700ms. Kamau's outbound calls now blocked at the SIP switch; data continues. He pays → SUB-WF-RESTRICT-01 (or SUB-WF-RESUME-01 depending on operator policy) calls FUL-04 with `LIFT_RESTRICTION` to reverse.

Alternative path — Adapter doesn't support restrictions

If the operator's VOIP adapter is a partner SIP service that doesn't implement `RestrictionCapableProvisioner`, the capability check (§3.3 R-CC-2) fails for VOIP. FUL-04 records `outcome = RESTRICTION_NOT_SUPPORTED_BY_PROVISIONER` for VOIP, skips dispatch. INET still dispatches (it's a different adapter). Terminal state may still be `FULLY_FULFILLED` if INET succeeded — but the restriction isn't actually enforced on VOIP. SUB-WF-RESTRICT-01 sees the per-Service outcome and decides whether to escalate (e.g., issue an operator alert) or accept the partial enforcement.

Alternative path — SUSPEND_SERVICE during package-change

When SUB-WF-MIGRATION-01 reaches its commit window and calls FUL-04 for source-side suspend, the envelope carries `intent=SUSPEND_SERVICE, reasonCategory=SOURCE_DECOMMISSION` (no `dunningEscalationLevel`). FUL-04 dispatches `voipSipProvisioner.suspendService(reasonCategory="SOURCE_DECOMMISSION", ...)` and `gponResProvisioner.suspendService(...)` — using the **base** `ServiceProvisioner.suspendService` method (not the restriction extension). The source-side service is taken offline; the workflow then proceeds to its source disconnect WO (via WO-01 SHIFTING flow), target install WO, target activate via FUL-03, target FUL-07 CLAIM.

5. Kafka events

Event	Topic	Fires on
`RestrictionFulfillmentStarted`	`sophix.fulfillment.restriction.started`	Dispatch begins (after ledger init)
`RestrictionFulfillmentCompleted`	`sophix.fulfillment.restriction.completed`	Terminal state `FULLY_FULFILLED` / `PARTIALLY_FULFILLED`
`RestrictionFulfillmentFailed`	`sophix.fulfillment.restriction.failed`	Terminal state `FAILED_AT_FIRST_SERVICE` / `FAILED_AND_ROLLED_BACK`
`CompensationFailed`	`sophix.fulfillment.restriction.compensationfailed`	Compensation invocation failed mid-rollback

All events carry `fulfillmentCorrelationId` for cross-system correlation.

Sample `RestrictionFulfillmentCompleted`:

```
{
  "eventType":   "RestrictionFulfillmentCompleted",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "timestamp":   "2026-05-15T10:32:42+03:00",
  "payload": {
    "fulfillmentId":             "fl4b_01HZ_KAMAU_APPLY",
    "operatorCode":              "WIK",
    "subscriptionId":            "sub_01HZ_KAMAU_FIBER",
    "intent":                    "APPLY_RESTRICTION",
    "restrictionCode":           "OUTGOING_VOICE_BARRED",
    "terminalState":             "FULLY_FULFILLED",
    "serviceOutcomes": [
      {"serviceRef":"svc_INET_100M","outcome":"SKIPPED_NOT_APPLICABLE"},
      {"serviceRef":"svc_VOIP_STD","outcome":"SUCCEEDED","adapterRefs":{"sipBarringId":"bar_77123"}}
    ],
    "fulfillmentCorrelationId":  "subop_01HZ_KAMAU_RESTRICT:apply",
    "callerWorkflowKind":        "SUB_WF_RESTRICT",
    "callerWorkflowRefId":       "subop_01HZ_KAMAU_RESTRICT",
    "completedAt":               "2026-05-15T10:32:42+03:00"
  }
}
```

6. Cross-DD coordination

Module	Direction	Contract
SUB-WF-RESTRICT-01	Caller	`ful-call-restrict` worker; `intent=APPLY_RESTRICTION` / `LIFT_RESTRICTION`
SUB-WF-SUSPEND-NP-01	Caller	`ful-call-suspend` worker; `intent=SUSPEND_SERVICE` + `reasonCategory=NON_PAYMENT` + dunning level

Module	Direction	Contract
SUB-WF-UPGRADE-01 / DOWNGRADE-01 / MIGRATION-01 / RELOCATION-01	Caller (source-side suspend during commit)	`ful-call-suspend` worker; `intent=SUSPEND_SERVICE` + `reasonCategory=SOURCE_DECOMMISSION`
FUL-03 v1.1	Sibling	Owns `ACTIVATE_SERVICE` / `MODIFY_SERVICE` / `RESUME_SERVICE`. FUL-03's `RESUME_SERVICE` is the legitimate post-pause / post-suspend restoration entry point — NOT a FUL-04 verb.
FUL-05	Sibling	Owns termination network revocation.
FUL-07	Sibling	Owns HomePass CLAIM / RELEASE bookkeeping.
PLM-CFG-01 v1.0	Reads Service catalog + Provisioner interface declarations	`ServiceProvisioner` (5 methods) + `RestrictionCapableProvisioner` (2 methods extension).
SIP-01 v1.0	Reads Package definitions	Service composition + pinned version
RLM-CFG-01 v1.0	Reads HomePass	`service_management_endpoints` + `status_code`
Provisioner adapters	Implementers	Implement the Provisioner interfaces per PLM-CFG-01 v1.0 §3. Adapters supporting restrictions implement `RestrictionCapableProvisioner` (§3.4); others implement only `ServiceProvisioner` (§3.1).
NOT-01	Indirect (via SUB-WF-RESTRICT-01)	Customer notification post-FUL-04.

6.1 Build order

This DD (v1.1) deploys after:

1. FUL-04 v1.0 (already deployed; v1.1 is a backwards-compatible extension)
2. PLM-CFG-01 v1.0 (formal Provisioner interfaces)
3. SIP-01 v1.0, RLM-CFG-01 v1.0
4. SUB-WF-RESTRICT-01, SUB-WF-SUSPEND-NP-01 (callers already use v1.0 envelopes; v1.1 is non-breaking)

6.2 Stub debt + open coordination items

- **Per-operator restriction code catalogs** — KE WIK uses workshop-derived codes; WUG/WTZ/YASSN catalogs pending operator deep dives.
- **Per-operator Provisioner adapter registration** — adapters register per `(provisionerKey, operatorCode)`. Operator-specific adapter implementations land as operators' deep-dives surface their vendor stacks.

B — Current TZ

Status: To be filled in after codebase cartography review.

A restriction-fulfillment implementation likely exists in the TZ codebase (the v1.0 baseline was authored against partial observation). Specific structural details — whether existing code separates `ServiceProvisioner` from `RestrictionCapableProvisioner`, whether SUSPEND_SERVICE is a base interface method or a restriction-extension method, whether operator scoping exists — require source-code review to document accurately.